

BANCO DE DADOS – TRANSAÇÕES

Profa. Vandecia Fernandes – BiCT/ECP/UFMA

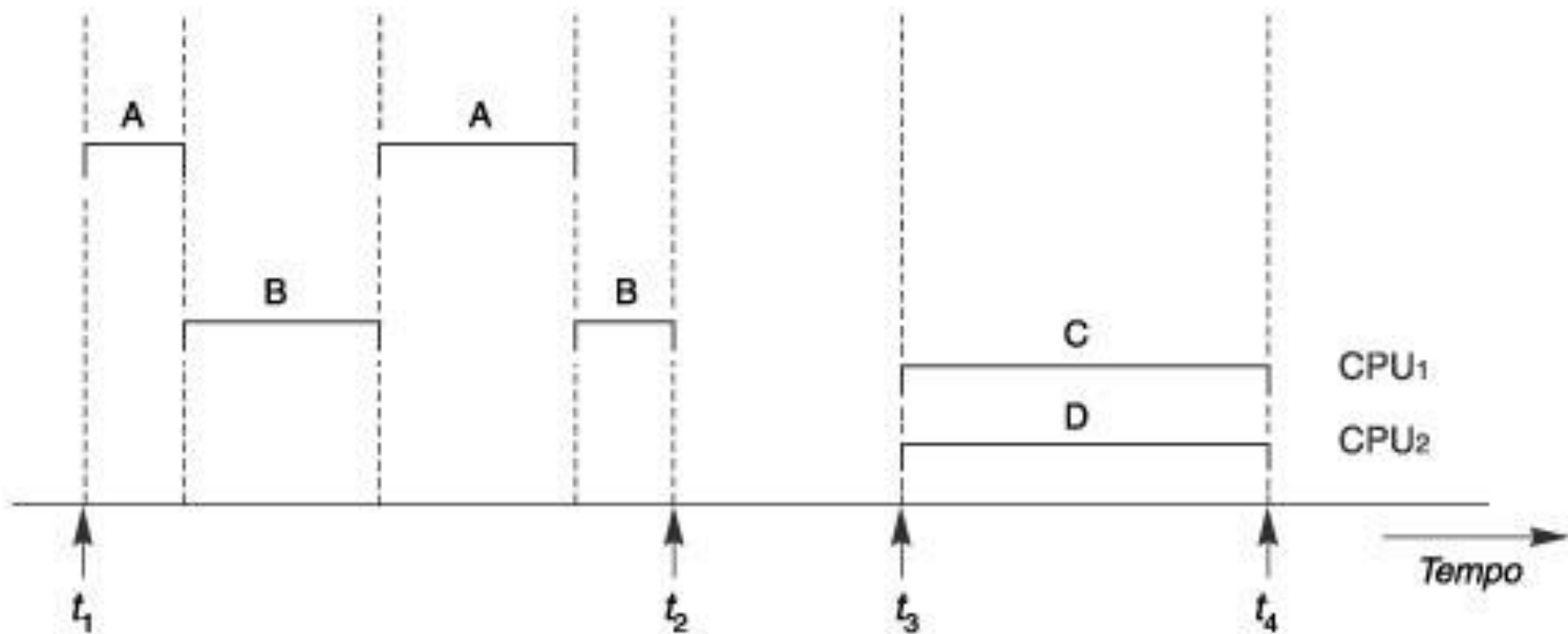
Referências: Elmasri, R. and Navathe, S.B. Sistemas de Bancos de Dados. Korth, H.F. e Silberschatz, A. Sistemas de Bancos de Dados.



EVOLUÇÃO DOS SISTEMAS

- Sistemas Monousuários
- Sistemas Multiusuários:
 - Compartilhamento de recursos (necessidade de controle de concorrência)

EVOLUÇÃO DOS SISTEMAS



TRANSAÇÕES

- Transações são sequências de ações elementares que deverão ser executadas como se fossem uma unidade atômica, sem interferência externa.
- Envolve múltiplas operações de acesso a um BD: operações de inserção, exclusão, alteração ou recuperação.
- As operações de BD que formam uma transação podem estar embutidas em linguagem de programação ou escritas em SQL

TRANSAÇÕES

- Um SGBD precisa:
- Garantir que a execução da transação seja completa;
- Administrar a execução simultânea de várias transações evitando inconsistências;

TRANSAÇÕES

- Exemplos de transações:
 - Saque bancário
 - Transferência de fundos
 - Reserva de passagem
 - Venda ao consumidor
 - Comércio entre empresas
 - Cadastro de usuário
 - Mudança de senha
 - Alocação de recurso
 - Controle de estoque

TRANSAÇÕES

■ Exemplo:

T_1

read(x);
 $x := x - N$;
write(x);
read(y);
 $y := y + N$;
write(y);

T_2

read(x);
 $x := x + M$;
write(x);

TRANSAÇÕES

- Exemplo: Seja T1 uma transação que transfere 50 reais da conta A para a conta B. Essa transação pode ser definida como:

```
T1: read(A);  
    A := A - 50;  
    write(A);  
    read(B);  
    B := B + 50;  
    write(B);
```

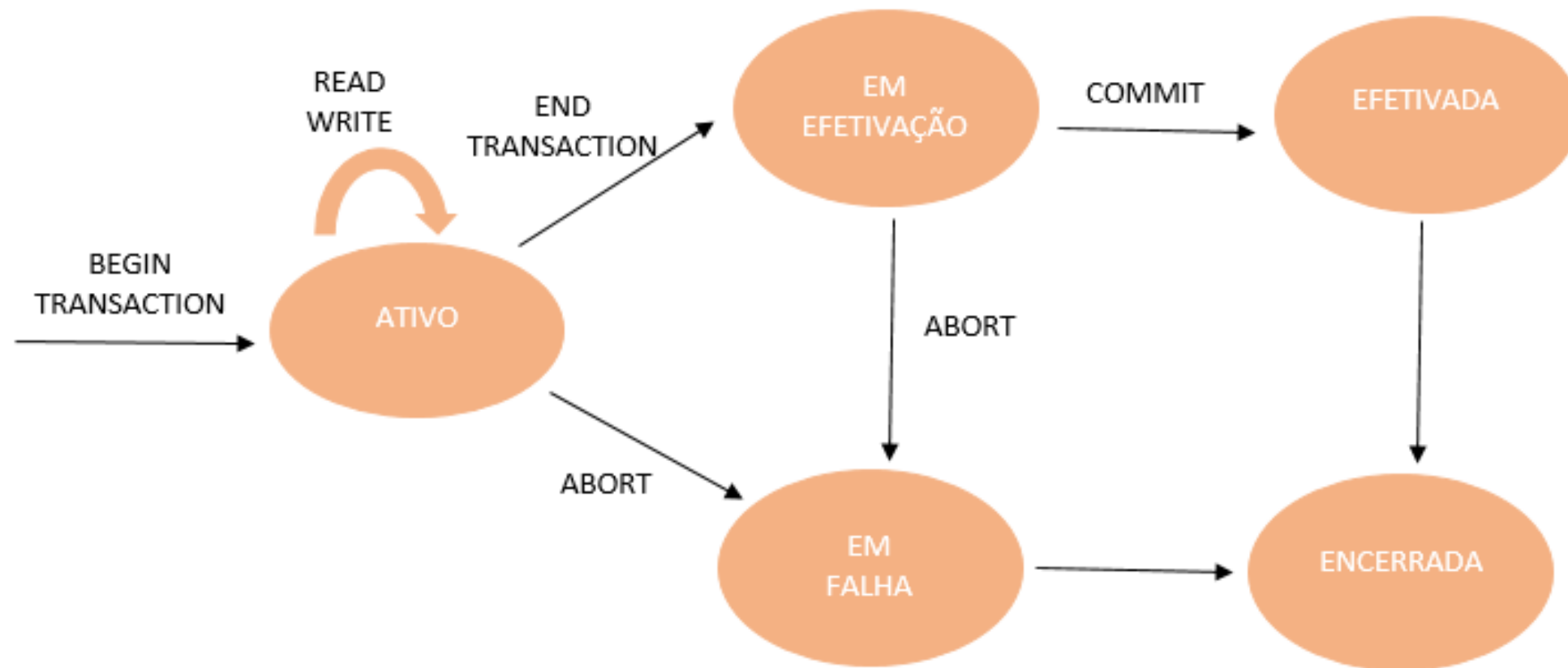

TRANSAÇÕES

- Operações:
- **Begin-transaction:** Denota o início da execução da transação;
- **End-transaction:** Especifica que as operações da transação terminaram e marca o limite final da execução da transação. Neste ponto é necessário verificar se o COMMIT ou o ABORT são necessários, caso já não tenham sido explicitados;
- **Commit-transaction:** Sinal de término com sucesso e que as alterações podem ser permanentemente gravadas no BD;

TRANSAÇÕES

- Operações:
- **Commit-transaction:** Sinal de término com sucesso e que as alterações podem ser “permanentemente” gravadas no BD;
- **Rollback (abort-transaction):** Assinala que a transação não terminou com sucesso e que seus efeitos devem ser desfeitos;
- **Undo:** Desfaz uma operação;
- **Redo:** Refaz uma operação.

ESTADOS DE UMA TRANSAÇÃO



ESTADOS DE UMA TRANSAÇÃO

- Uma transação é concluída ao ser efetivada ou abortada pelo sistema
- Ao ser efetivada, a transação só pode ser desfeita com uma transação de compensação

ESTADOS DE UMA TRANSAÇÃO

- Opções em caso de aborto da transação:
 - Reiniciar a transação: se a falha for transitória
 - Matar a transação: se a falha for permanente
- Recuperação de Estado (Rollback):
 - Ocorre quando a transação é abortada: Dados retomam os valores que tinham antes do início da transação

TIPOS DE FALHAS

- Tipos de falhas:
 - Falha de processamento: ocorrida ao processar a transação devido a dois tipos de erro:
 - Erro lógico: transação interrompida devido a uma entrada inválida, dado inexistente, overflow, etc.
 - Erro de sistema: transação é abortada ao atingir um estado inadequado (ex.: deadlock)

TIPOS DE FALHAS

- Tipos de falhas:
 - Falha de dispositivo: ocorrida devido a um bloco de disco corrompido ou a um defeito irreparável no HD.
 - Falha de sistema: causada por bugs no software de banco de dados ou no sistema operacional, ou por uma falha no hardware.

PROPRIEDADES DAS TRANSAÇÕES (ACID)

- **ACID**

- **A**tomicidade
- **C**onsistência
- **I**solamento
- **D**urabilidade

PROPRIEDADES DAS TRANSAÇÕES (ACID)

- **ACID**
 - **A**tomicidade
 - **C**onsistência
 - **I**solamento
 - **D**urabilidade

PROPRIEDADES DAS TRANSAÇÕES (ACID)

- Atomicidade:
 - Ou todas as operações da transação são refletidas corretamente no banco de dados ou nenhuma o será.
- Assegurar a atomicidade de uma transação é responsabilidade do SGBD, mais especificamente, do Componente de Gerenciamento de Transações e Recuperador de Falhas.

PROPRIEDADES DAS TRANSAÇÕES (ACID)

- **ACID**

- **A**tomicidade
- **C**onsistência
- **I**solamento
- **D**urabilidade

PROPRIEDADES DAS TRANSAÇÕES (ACID)

- Consistência:
 - A execução completa da transação faz o banco de dados passar de um estado consistente para outro.
- Responsabilidade do programador.

PROPRIEDADES DAS TRANSAÇÕES (ACID)

- **ACID**

- **A**tomicidade
- **C**onsistência
- **I**solamento
- **D**urabilidade

PROPRIEDADES DAS TRANSAÇÕES (ACID)

- Isolamento:
 - A execução de uma transação não deve sofrer interferência de quaisquer outras transações concorrentes.
- Assegurar o isolamento é de responsabilidade do Controlador de Concorrência do SGBD

PROPRIEDADES DAS TRANSAÇÕES (ACID)

- **ACID**

- **A**tomicidade
- **C**onsistência
- **I**solamento
- **D**urabilidade

PROPRIEDADES DAS TRANSAÇÕES (ACID)

- Durabilidade
 - As mudanças aplicadas ao banco de dados por uma transação efetivada devem persistir no banco de dados, mesmo em caso de falhas.
- Assegurar a durabilidade é responsabilidade do componente do SGBD chamado de Recuperador de Falhas.

PROPRIEDADES DAS TRANSAÇÕES (ACID)

- Exemplo de transação (aplicação financeira):

Read(Conta);

Conta.Saldo = Conta.Saldo – 100;

Write(Conta);

Read (Aplicação);

Aplicação.Saldo = Aplicação.Saldo + 100;

Write (Aplicação);

- Aplicação das propriedades ACID:
- Atomicidade: faz todas as operações ou nenhuma
- Consistência: deve manter o saldo total do cliente
- Isolamento: controla as operações concorrentes
- Durabilidade: salva as modificações no BD

PROPRIEDADES DAS TRANSAÇÕES (ACID)

- Transações podem ser executadas de modo:
- **Sequencial:**
 - uma transação T_i termina antes do início ou começa após o final de uma transação T_j
- **Concorrente:**
 - partes das transações T_i e T_j podem ser processadas simultaneamente

CONCORRÊNCIA

- Concorrência entre transações é necessária para melhorar o desempenho:
- Aproveita o tempo que uma transação permanece bloqueada devido a operações de leitura e escrita no disco para processar outras transações.(Alternância entre as transações)
- Evita que transações muito longas retardem a execução de transações mais curtas.

CONCORRÊNCIA

- Execuções concorrentes precisam garantir as propriedades de consistência e isolamento:
- A definição de escalas de execução (*schedules*) identifica as operações que podem ser efetuadas concorrentemente sem afetar a consistência do BD.
- O Controle de concorrência impede que um dado que está sendo alterado por uma transação seja usado por uma outra transação.

CONCORRÊNCIA

- Exemplo: Considere duas transações T1 e T2:
- Suponha que o cliente tem um saldo total de R\$2000, sendo R\$1000 na conta e R\$1000 em aplicações:

CONCORRÊNCIA

- T1: Read (Conta);
Conta.Saldo = Conta.Saldo – 200;
Write (Conta);
Read (Aplic);
Aplic.Saldo = Aplic.Saldo + 200;
Write (Aplic);
- T2: Read (Conta);
 $X = \text{Conta.Saldo} * 0.5$;
Conta.Saldo = Conta.Saldo – X;
Write (Conta);
Read (Aplic);
Aplic.Saldo = Aplic.Saldo + X;
Write (Aplic);

CONCORRÊNCIA

- Se T1 for executada antes de T2 teremos ao final
 - Conta.Saldo = R\$ 400
 - Aplic.Saldo = R\$ 1600
- Já se T2 for executada antes de T1, teremos
 - Conta.Saldo = R\$ 300
 - Aplic.Saldo = R\$ 1700
- O saldo total é mantido em R\$ 2000 nos dois casos

CONCORRÊNCIA

- Exemplo de escala concorrente equivalente à execução sequencial $T_1 \rightarrow T_2$

T_1	T_2
Read (Conta); Conta.Saldo = Conta.Saldo - 200; Write (Conta);	Bloqueado
Bloqueado	Read (Conta); $X = \text{Conta.Saldo} * 0.5$; Conta.Saldo = Conta.Saldo - X; Write (Conta);
Read (Aplic); Aplic.Saldo = Aplic.Saldo + 200; Write (Aplic);	Bloqueado
Bloqueado	Read (Aplic); Aplic.Saldo = Aplic.Saldo + X; Write (Aplic);

CONCORRÊNCIA

- Exemplo de escala concorrente que não respeita o isolamento: saldo total não é preservado

T_1	T_2
Read (Conta); Conta.Saldo = Conta.Saldo - 200;	Bloqueado
Bloqueado	Read (Conta); $X = \text{Conta.Saldo} * 0.5$; Conta.Saldo = Conta.Saldo - X; Write (Conta);
Write (Conta); Read (Aplic); Aplic.Saldo = Aplic.Saldo + 200; Write (Aplic);	Bloqueado
Bloqueado	Read (Aplic); Aplic.Saldo = Aplic.Saldo + X; Write (Aplic);

CONCORRÊNCIA

- Exemplo de escala concorrente que não respeita o isolamento: saldo total não é preservado

T_1	T_2
Read (Conta); Conta.Saldo = Conta.Saldo - 200;	Bloqueado
Bloqueado	Read (Conta); $X = \text{Conta.Saldo} * 0.5$; Conta.Saldo = Conta.Saldo - X; Write (Conta);
Write (Conta); Read (Aplic); Aplic.Saldo = Aplic.Saldo + 200; Write (Aplic);	Bloqueado
Bloqueado	Read (Aplic); Aplic.Saldo = Aplic.Saldo + X; Write (Aplic);

CONCORRÊNCIA

- A recuperação de estado é dificultada quando há concorrência entre transações:
- Na escala concorrente do slide 26, se T1 falha, T2 tem que ser refeita, pois leu um valor modificado por T1
- Nestes casos, a recuperação é feita em cascata: cancelar as operações que leram valores inválidos

CONCORRÊNCIA

- A recuperação de estado é dificultada quando há concorrência entre transações:
- Rollback em cascata pode ser evitado obrigando que T2 leia o valor escrito por T1 só após o commit de T1
- Se a efetivação de T2 ocorrer antes do commit de T1 será impossível fazer o rollback em cascata

CONCORRÊNCIA - PROBLEMAS

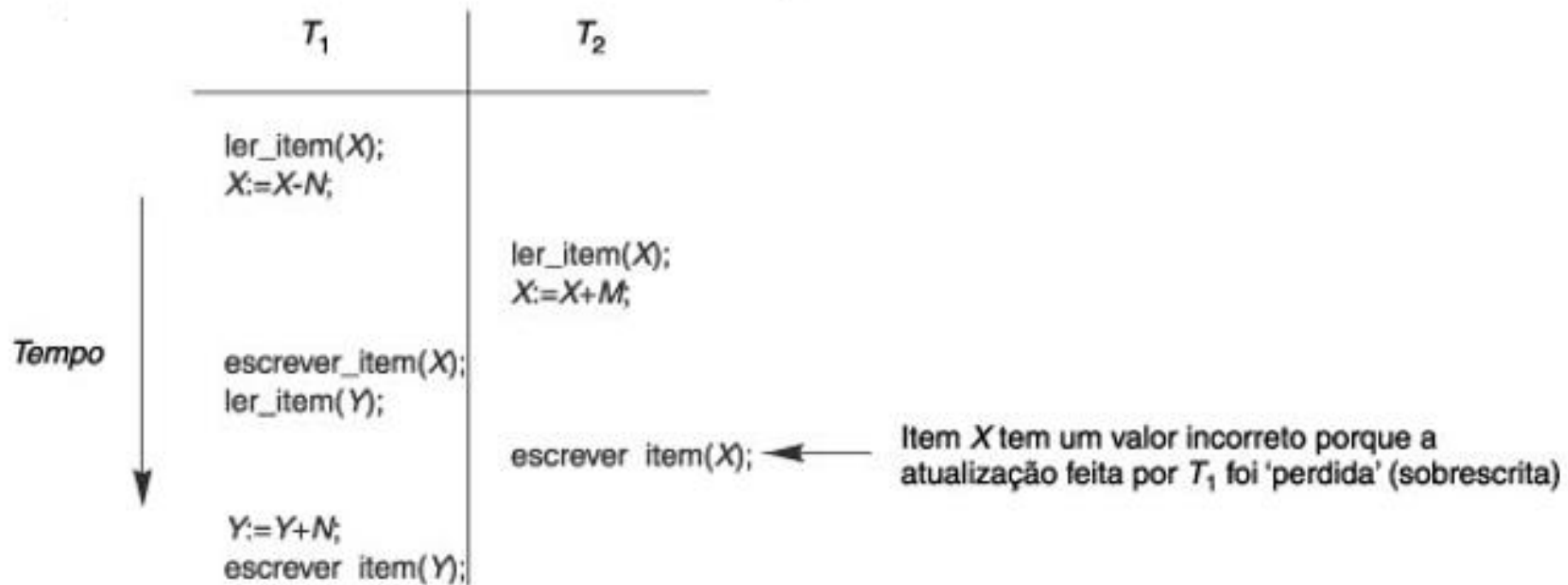
- Problemas na Concorrência:
 - Problema de alterações perdidas
 - Problema de Alteração Temporária (*Dirty Read*)
 - Problema do resumo(sumário) incorreto

CONCORRÊNCIA - PROBLEMAS

- Problemas na Concorrência:
 - Problema de alterações perdidas:
 - Ocorre quando duas transações que acessam o mesmo item de BD possuem suas operações intercaladas de uma maneira que o valor de algum item de dado fique incorreto.

CONCORRÊNCIA - PROBLEMAS

- Problema de alterações perdidas

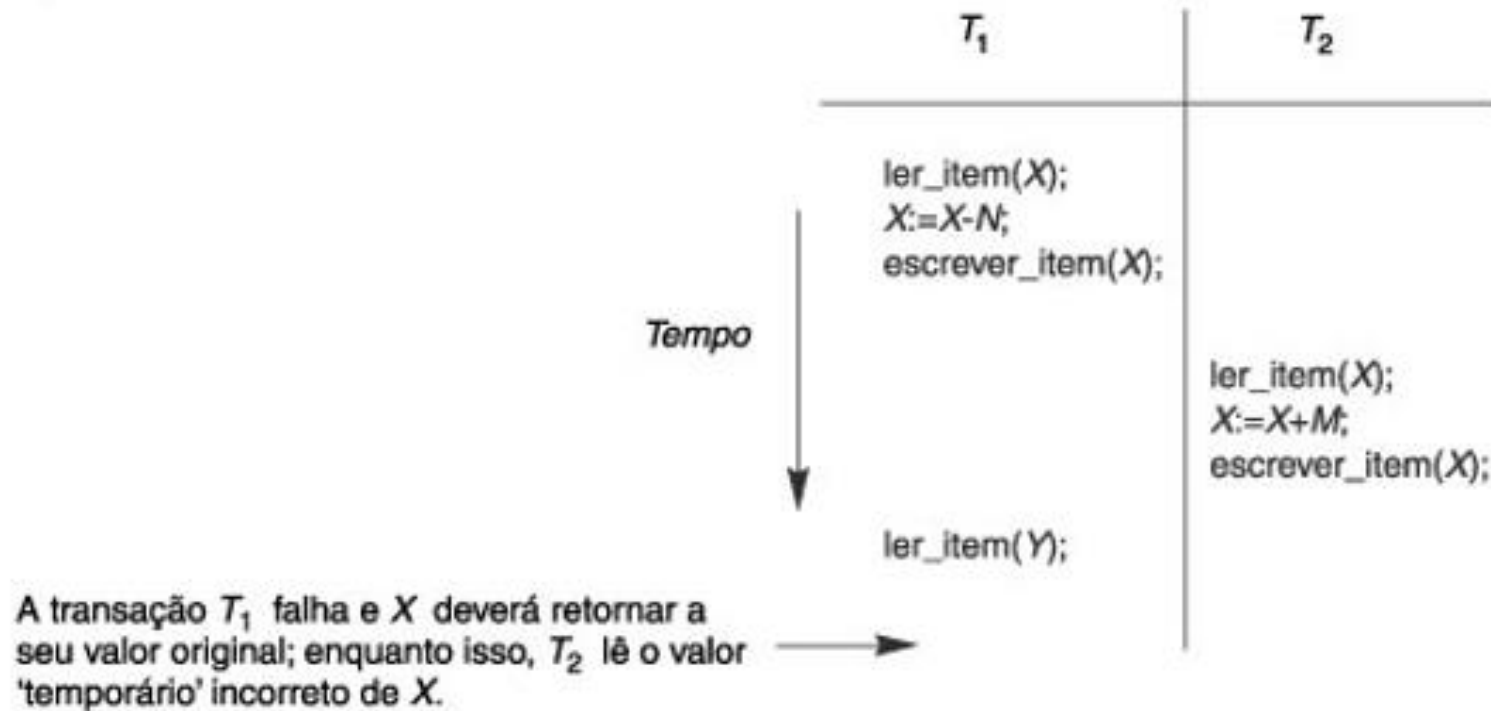


CONCORRÊNCIA - PROBLEMAS

- Problemas na Concorrência:
- Problema de Alteração Temporária (*Dirty Read*):
- Ocorre quando uma transação altera um item de dados e depois ela falha por alguma razão. O item de dado é acessado por outra transação antes que o valor original seja estabelecido.

CONCORRÊNCIA - PROBLEMAS

- Problema de alteração temporária

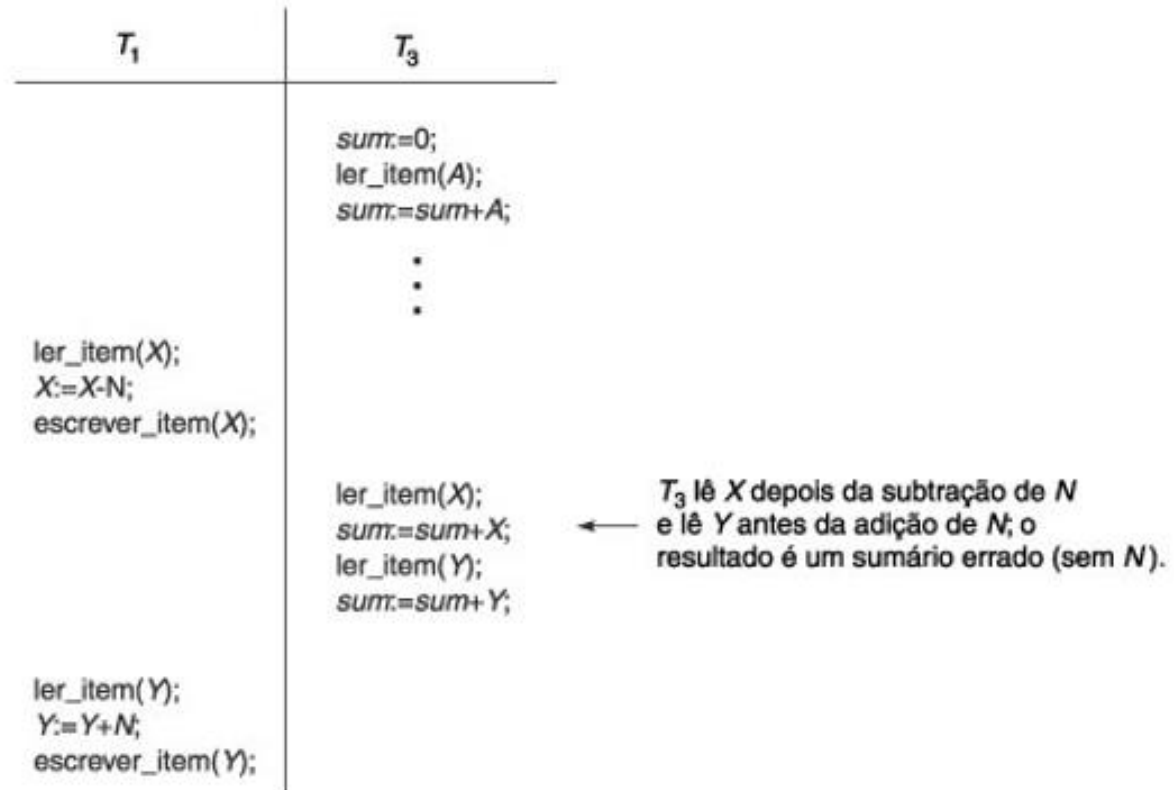


CONCORRÊNCIA - PROBLEMAS

- Problemas na Concorrência:
- Problema do resumo(sumário) incorreto:
 - Se uma transação está calculando uma função agregada com um conjunto de tuplas e outras transações estão alterando algumas destas tuplas, a função agregada pode calcular alguns valores antes deles serem alterados e outros depois de serem alterados.

CONCORRÊNCIA - PROBLEMAS

○ Problema do resumo incorreto



SERIALIZAÇÃO

- O sistema de banco de dados deve controlar a execução concorrente de transações para assegurar que o estado do BD permaneça consistente.
- A consistência do banco de dados, sob execução concorrente, pode ser assegurada por meio da garantia de que qualquer escala executada tenha o mesmo efeito de outra que tivesse sido executada sem qualquer concorrência.

SERIALIZAÇÃO

- Uma escala concorrente equivalente a uma escala sequencial é dita serializável.
- Formas de serialização de escalas de execução:
 - Serialização de Conflito
 - Serialização de Visão

SERIALIZAÇÃO DE CONFLITO

- Instruções das transações T_i e T_j são conflitantes se ambas acessam um mesmo dado Q e se pelo menos uma delas modifica o valor de Q
- Se duas instruções não são conflitantes, sua ordem pode ser trocada na escala

Instrução de T_i	Instrução de T_j	Há conflito?
Read(Q)	Read(Q)	Não
Read(Q)	Write(Q)	Sim
Write(Q)	Read(Q)	Sim
Write(Q)	Write(Q)	Sim

SERIALIZAÇÃO DE CONFLITO

T_1	T_2
read(A) write(A)	
	read(A) write(A)
read(B) write(B)	
	read(B) write(B)

A instrução `write(A)` de T_1 entra em conflito com a instrução `read(A)` de T_2 .

Porém, a instrução `write(A)` e T_2 não está em conflito com a instrução `read(B)` de T_1 .

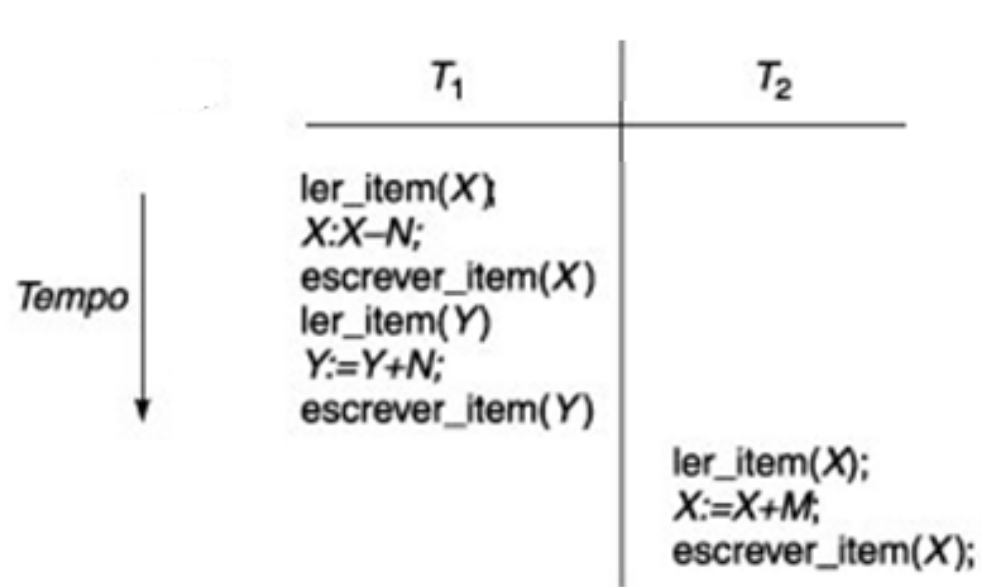
SERIALIZAÇÃO DE CONFLITO

- Se uma escala de execução S puder ser transformada em outra, S' , por uma série de trocas de instruções não conflitantes, dizemos que S e S' são equivalentes por conflito.

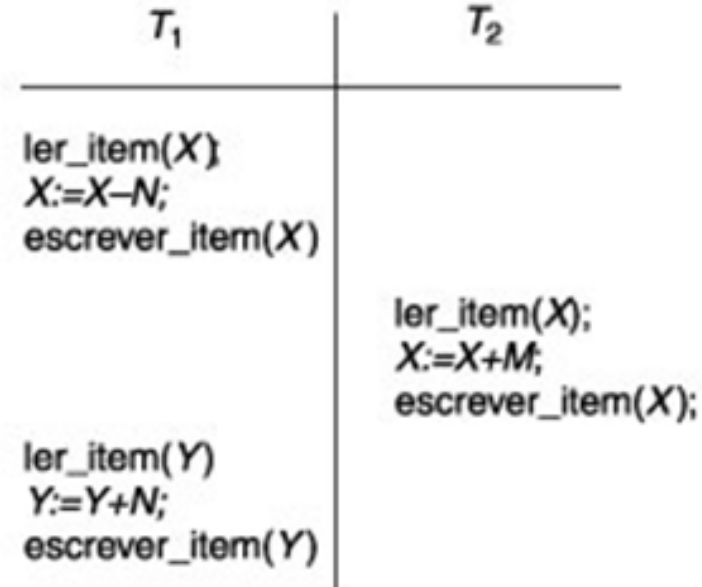
SERIALIZAÇÃO DE CONFLITO

- O conceito de equivalência por conflito leva ao conceito de serialização por conflito.
- Uma escala de execução S é serializável por conflito se ela é equivalente por conflito a uma escala de execução seqüencial.

SERIALIZAÇÃO DE CONFLITO



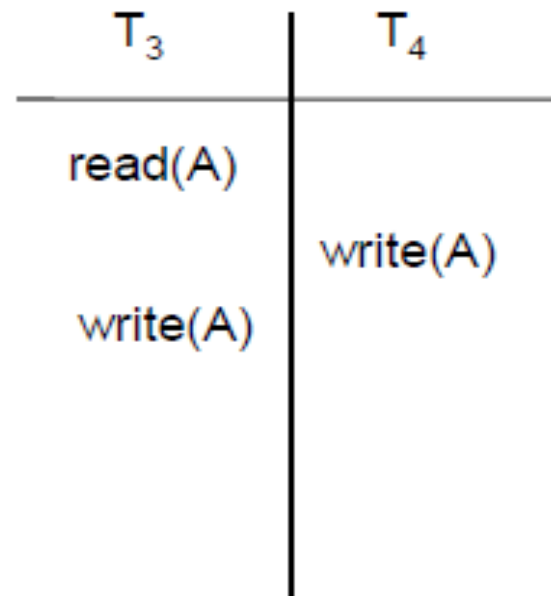
PLANO S



PLANO S'

SERIALIZAÇÃO DE CONFLITO

- A escala abaixo não é serializável por conflito pois não é equivalente em conflito nem à escala seqüencial $\langle T_3, T_4 \rangle$ nem $\langle T_4, T_3 \rangle$.



SERIALIZAÇÃO DE CONFLITO

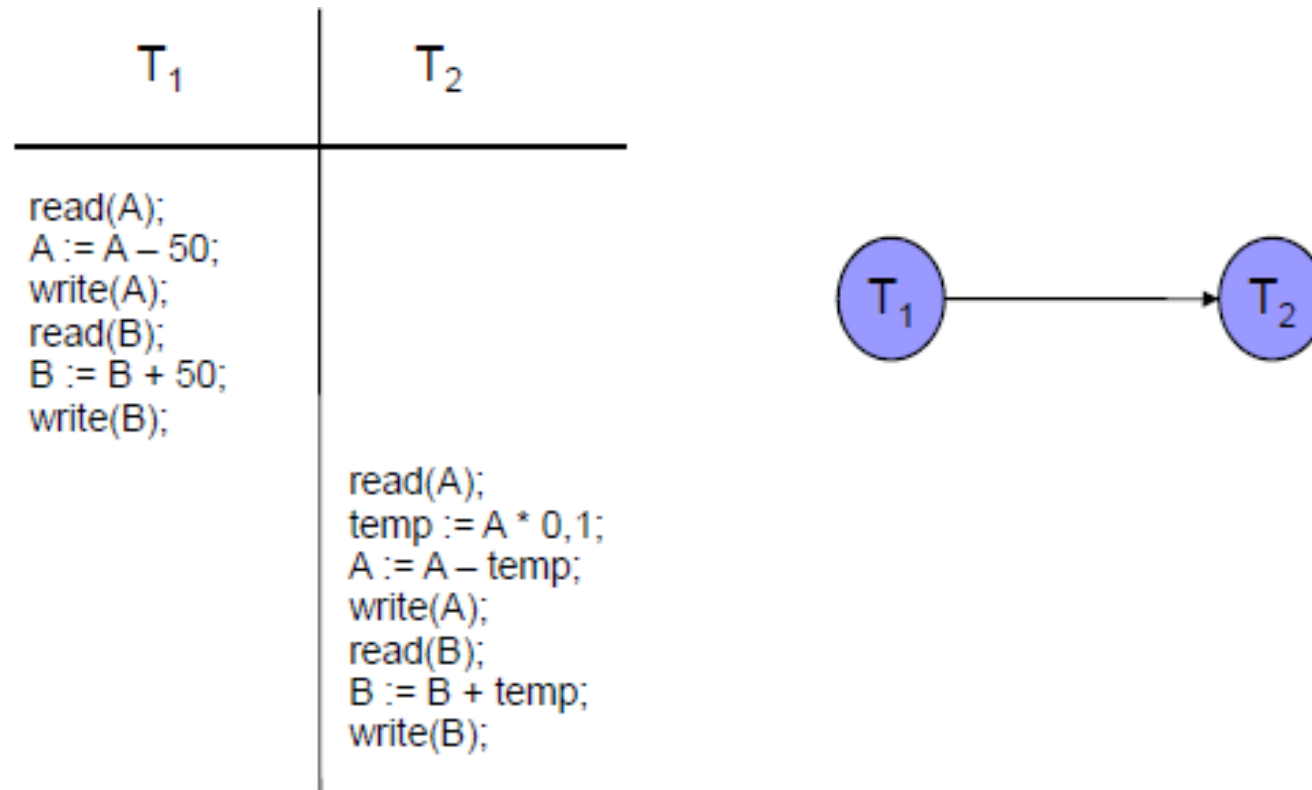
- Seja S uma escala. Para saber se ela é serializável em relação às operações conflitantes é necessário criar um grafo de precedência para S .
- $G = (V, E)$ em que V é um conjunto de vértices e E é um conjunto de arestas.
 - O conjunto de vértices é composto por todas as transações que participam da escala.

SERIALIZAÇÃO DE CONFLITO

- O conjunto de arestas consiste em todas as arestas $T_i \rightarrow T_j$ para as quais uma das seguintes condições é verdadeira:
 - T_i executa $\text{write}(Q)$ antes de T_j executar $\text{read}(Q)$;
 - T_i executa $\text{read}(Q)$ antes de T_j executar $\text{write}(Q)$;
 - T_i executa $\text{write}(Q)$ antes de T_j executar $\text{write}(Q)$;

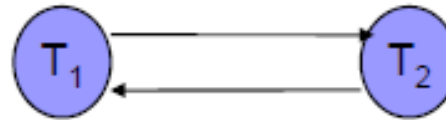
SERIALIZAÇÃO DE CONFLITO

- Se existir uma aresta $T_i \rightarrow T_j$ no grafo de precedência, então, em qualquer escala seqüencial S' equivalente a S , T_i deve aparecer antes de T_j .



SERIALIZAÇÃO DE CONFLITO

T_1	T_2
read(A) $A := A - 50$	read(A) $temp := A * 0,1$ $A := A - temp$ write(A) read(B)
write(A) read(B) $B := B + 50$ write(B)	$B := B + temp$ write(B)



Ciclo no grafo: Se o grafo de precedência possui ciclo, então a escala S não é serializável por conflito.

A ordem de serialização pode ser obtida por meio da classificação topológica, que estabelece uma ordem linear para a escala consistente com a ordem parcial do grafo de precedência.

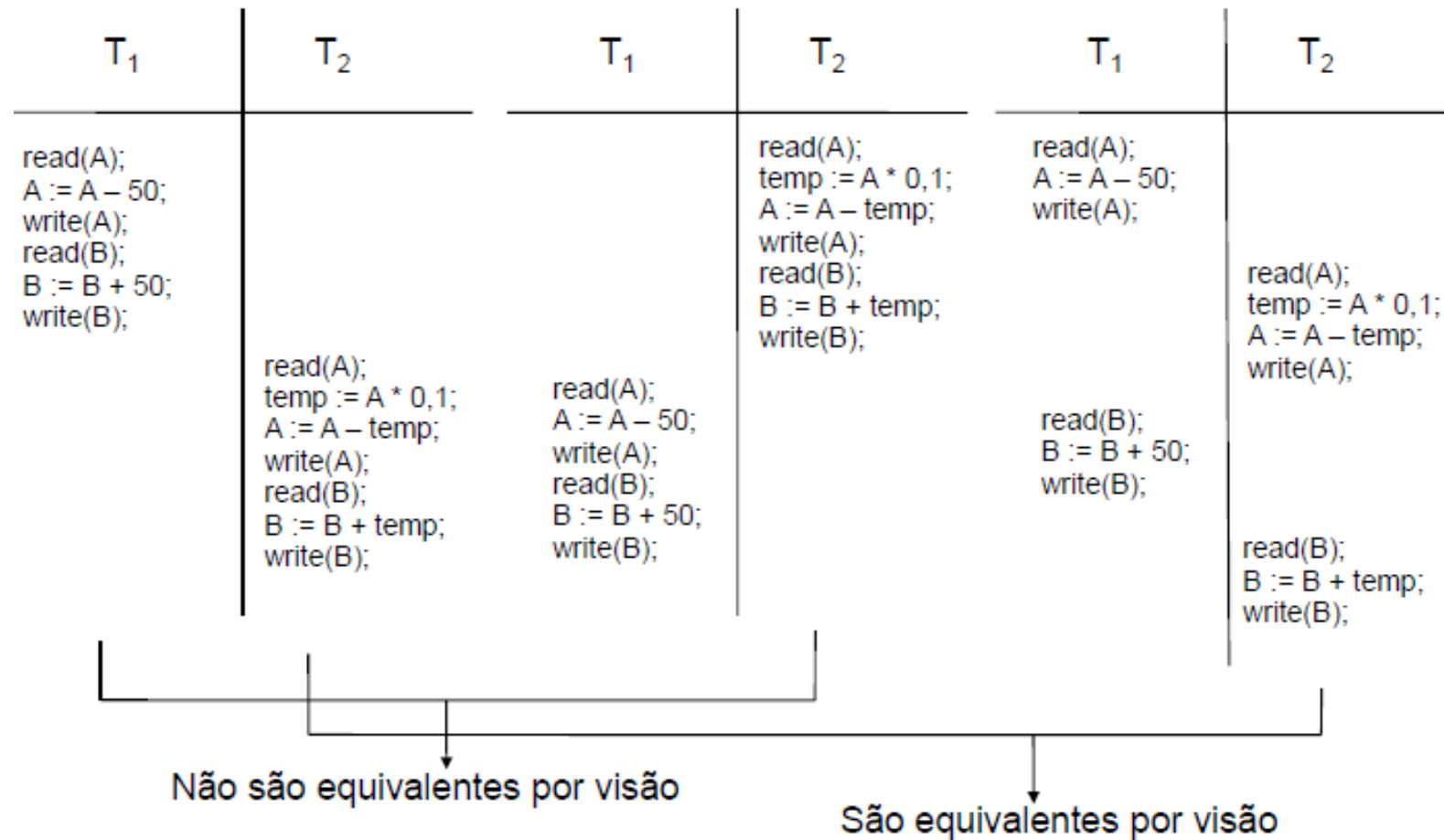
EQUIVALENCIA EM VISÃO

- Trata-se de uma forma de equivalência menos restritiva que a equivalência de conflito.

EQUIVALÊNCIA EM VISÃO

- As seguintes regras devem ser respeitadas para que duas escalas S e S' sejam equivalentes em visão:
 - Se na escala S a transação T_i lê o valor inicial de um dado, o mesmo deve ocorrer na escala S'
 - Se na escala S a transação T_i lê o valor de um dado escrito por T_j , o mesmo deve ocorrer em S'
 - Se na escala S a transação T_i escreve o valor final de Q , o mesmo deve ocorrer em S'

EQUIVALÊNCIA EM VISÃO



SERIALIZAÇÃO DE VISÃO

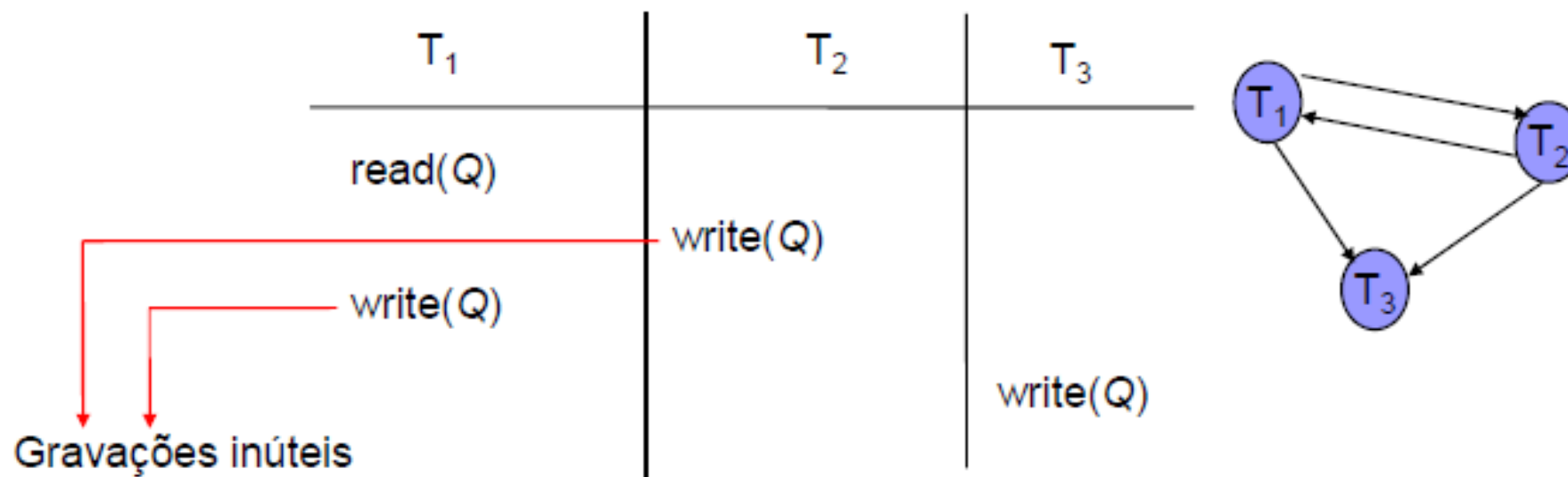
- O conceito de equivalência por visão leva ao conceito de serialização por visão.
- Uma escala é serializável por visão se ela for equivalente em visão a uma escala de execução sequencial.

SERIALIZAÇÃO DE VISÃO

- Modifica-se o grafo de precedência para serialização por conflito para testar se o escalonamento é serializável por visão.
- Escalas serializáveis em conflito são sempre serializáveis em visão, mas não o contrário.

EQUIVALÊNCIA EM VISÃO

- Verificando se o escalonamento abaixo é serializável por conflito:



SERIALIZAÇÃO

- Considerações Finais:
- Escalas **devem ser** serializáveis em conflito ou visão
- Alguns SGBD aceitam escalas serializáveis em visão, mesmo que haja conflito; outros permitem apenas escalas serializáveis em conflito

SERIALIZAÇÃO

- Considerações Finais:
- SGBDs podem ser configurados para aceitar menores níveis de consistência (ex.: permitir leitura de dados ainda não efetivados)
- Escalas devem ser recuperáveis, preferencialmente sem a necessidade de recuperação em cascata (quanto maior a concorrência, maior a possibilidade de termos que fazer recuperação em cascata)



DÚVIDAS?